

Introduction

Compiler ./. Interpreter



- Translates a program from one language into another
- Compilation may be slow, but can produce highly optimized code
- More checks before runtime
- Lines are blurred:
 - Interpreters may use **JIT (Just-In-Time)** compilation to optimize hot-code
 - Interpreters may compile into byte code
 - Compilers produce *byte code* that gets interpreted
- Executes a program to directly produce the effect
- Often easier to develop for
- More complex runtime environment

Compiler: Frontend ./ Backend

GCC (Gnu Compiler Collection) supports ...

- $n_{\text{Input}} = 11$ source languages: C, C++, Objective-C, D, Fortran, Ada, Go, Rust, Modula-2, COBOL, and Algol 68
- $n_{\text{Output}} > 40$ target languages: Alpha, Arm AArch32, Arm AArch64, H8/300, S/370, S/390, i386, x86-64, Itanium (IA-64), Motorola 68000 und Motorola Coldfire, Motorola 88000, MIPS-Architektur (primär unter IRIX), PA-RISC (primär unter HP-UX), PDP-11, PowerPC, RISC-V, SuperH, Sun SPARC, VAX, Motorola 68HC11, A29K, Adapteva Epiphany, Atmel AVR, Blackfin, C4x, CRIS, D30V, DSP16xx, FR-30, FR-V, Intel i960, IP2000, M32R, MCORE, MicroBlaze, MMIX, MN10200, MN10300, NS32K, ROMP, Stormy16, Synopsis Designware ARC, Texas Instruments MSP430, MSP432, V850, Xtensa

Compiler: Frontend ./ Backend

GCC (Gnu Compiler Collection) supports ...

- $n_{\text{Input}} = 11$ source languages: C, C++, Ada, Go, Rust, ...
- $n_{\text{Output}} > 40$ target languages: Arm AArch32, Arm AArch64, i386, x86-64, ...

Compiler: Frontend ./ Backend

GCC (Gnu Compiler Collection) supports ...

- $n_{\text{Input}} = 11$ source languages: C, C++, Ada, Go, Rust, ...
- $n_{\text{Output}} > 40$ target languages: Arm AArch32, Arm AArch64, i386, x86-64, ...

Adding a new target language is called **retargetting**.

Compiler: Frontend ./ Backend

GCC (Gnu Compiler Collection) supports ...

- $n_{\text{Input}} = 11$ source languages: C, C++, Ada, Go, Rust, ...
- $n_{\text{Output}} > 40$ target languages: Arm AArch32, Arm AArch64, i386, x86-64, ...

Adding a new target language is called **retargetting**.

- **Issue:** We cannot implement $n_{\text{Input}} \times n_{\text{Output}} > 400$ compilers! **Better:**

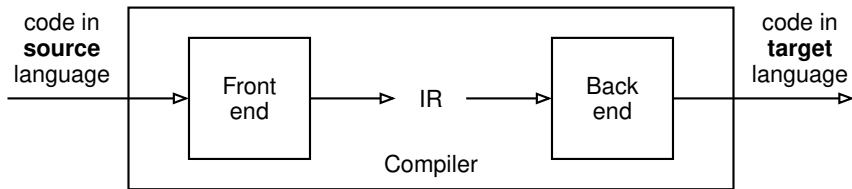
Compiler: Frontend ./ Backend

GCC (Gnu Compiler Collection) supports ...

- $n_{\text{Input}} = 11$ source languages: C, C++, Ada, Go, Rust, ...
- $n_{\text{Output}} > 40$ target languages: Arm AArch32, Arm AArch64, i386, x86-64, ...

Adding a new target language is called **retargetting**.

- **Issue:** We cannot implement $n_{\text{Input}} \times n_{\text{Output}} > 400$ compilers! **Better:**

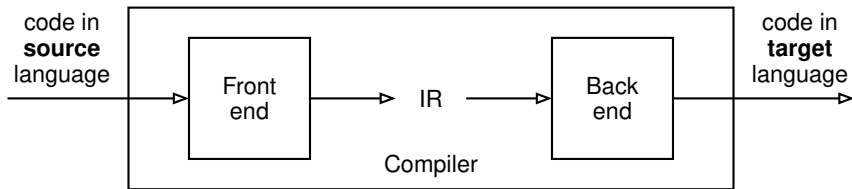


Compiler: Frontend ./ Backend

GCC (Gnu Compiler Collection) supports ...

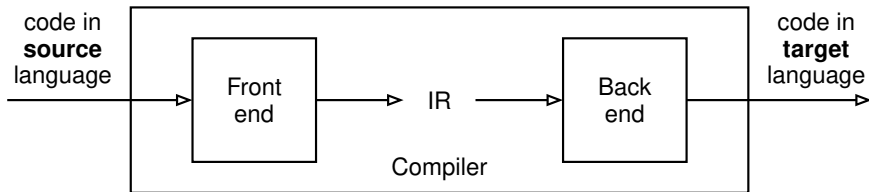
- $n_{\text{Input}} = 11$ source languages: C, C++, Ada, Go, Rust, ...
 - $n_{\text{Output}} > 40$ target languages: Arm AArch32, Arm AArch64, i386, x86-64, ...
- Adding a new target language is called **retargetting**.

- **Issue:** We cannot implement $n_{\text{Input}} \times n_{\text{Output}} > 400$ compilers! **Better:**



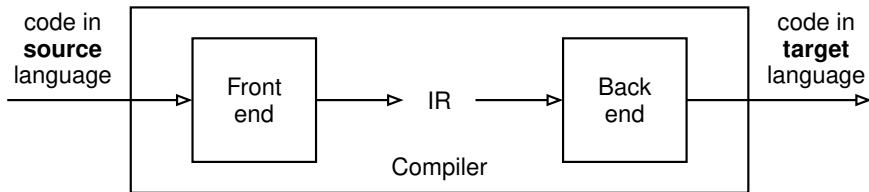
- “only” needs $n_{\text{Input}} + n_{\text{Output}} \approx 50$ languages

Two-phase compilation



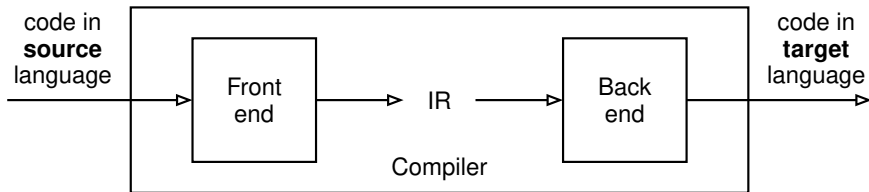
- **Intermediate representation (IR)** is the “glue” between multiple phases
 - better software design
 - concentrate on one task
 - more flexible

Two-phase compilation



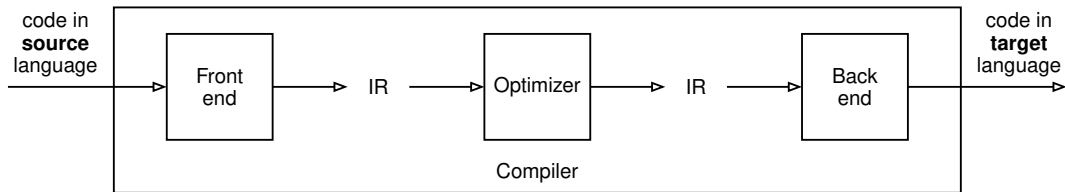
- **Intermediate representation (IR)** is the “glue” between multiple phases
 - better software design
 - concentrate on one task
 - more flexible
- Often assembly-like language
- Well known example: LLVM (IR) Language

Two-phase compilation



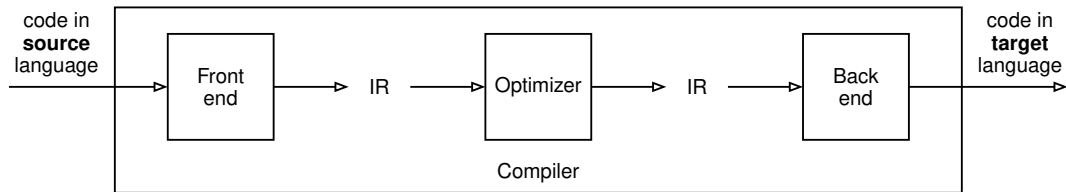
- **Intermediate representation (IR)** is the “glue” between multiple phases
 - better software design
 - concentrate on one task
 - more flexible
- Often assembly-like language
- Well known example: LLVM (IR) Language
- IR may or may not have a textual representation

Three phase compilation: **Optimizing compilers**



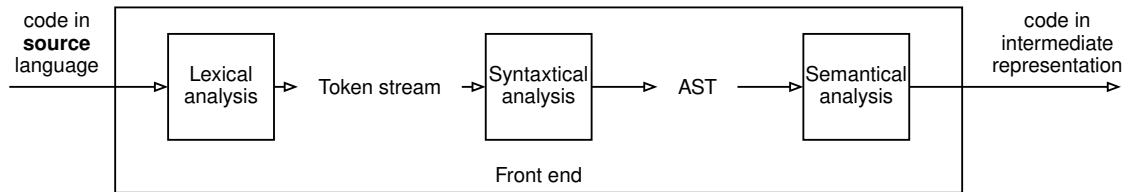
- Transform a program to a “better” program **in the same language**

Three phase compilation: **Optimizing compilers**



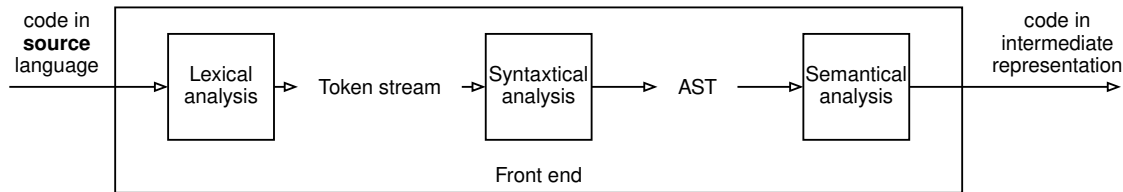
- Transform a program to a “better” program **in the same language**
 - The optimizer is technically a *compiler* itself!
- Many optimization can be done in a frontend/backend oblivious way:
 - Dead-code elimination
 - Data-flow opt (common subexpression elimination, constant folding, ...)
 - Loop-optimizations ...

The front end



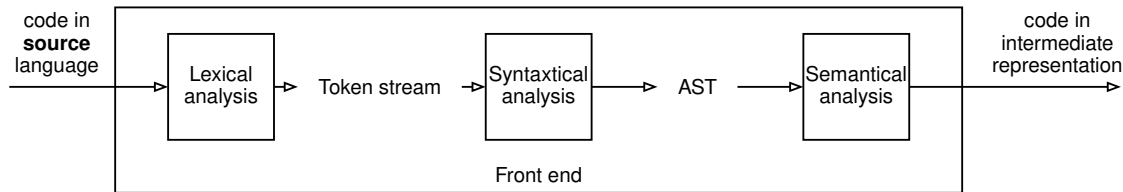
- **Lexical analysis** (aka **Scanner** or **Lexer**)
 - Identify “words” (e.g., keywords, identifiers, number literals)

The front end



- **Lexical analysis** (aka **Scanner** or **Lexer**)
 - Identify “words” (e.g., keywords, identifiers, number literals)
- **Syntactical analysis** (aka **Parser**)
 - Identify the “sentences” implied by the “words” according to grammar rules

The front end



- **Lexical analysis** (aka **Scanner** or **Lexer**)
 - Identify “words” (e.g., keywords, identifiers, number literals)
- **Syntactical analysis** (aka **Parser**)
 - Identify the “sentences” implied by the “words” according to grammar rules
- **Semantical analysis**
 - Type checking, etc.
 - Intermediate code generation

Lexer: Finding the right words

Input:

```
print( (a+3)  *b + (a+3)*c / 0x8 );
```

Output:

```
[]
```

Lexer: Finding the right words

Input:

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

Output:

```
[ Ident('print'), ]
```

Lexer: Finding the right words

Input:

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

Output:

```
[ Ident('print'), ParenLeft, ]
```

Lexer: Finding the right words

Input:

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

Output:

```
[ Ident('print'), ParenLeft, ignore whitespace, ]
```

Lexer: Finding the right words

Input:

```
print( (a+3)  *b + (a+3)*c / 0x8 );
```

Output:

```
[ Ident('print'), ParenLeft, ParenLeft, ]
```

Lexer: Finding the right words

Input:

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

Output:

```
[ Ident('print'), ParenLeft, ParenLeft, Ident('a'), ]
```

Lexer: Finding the right words

Input:

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

Output:

```
[ Ident('print'), ParenLeft, ParenLeft, Ident('a'), Plus, ]
```

Lexer: Finding the right words

Input:

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

Output:

```
[ Ident('print'), ParenLeft, ParenLeft, Ident('a'), Plus, Num(3), ]
```

Lexer: Finding the right words

Input:

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

Output:

```
[ Ident('print'), ParenLeft, ParenLeft, Ident('a'), Plus, Num(3), ParenRight, ]
```

Lexer: Finding the right words

Input:

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

Output:

```
[ Ident('print'), ParenLeft, ParenLeft, Ident('a'), Plus, Num(3), ParenRight,  
ignore whitespace, Times, Ident('b'), ignore whitespace, Plus, ignore whitespace,  
ParenLeft, Ident('a'), Plus, Num(3), ParenRight, Times, Ident('c'), ignore  
whitespace, Div, ignore whitespace, Num(8), ignore whitespace, ParenRight,  
Semicolon, ]
```

Lexer: Finding the right words

Input:

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

Output:

```
[ Ident('print'), ParenLeft, ParenLeft, Ident('a'), Plus, Num(3), ParenRight,  
Times, Ident('b'), Plus, ParenLeft, Ident('a'), Plus, Num(3), ParenRight,  
Times, Ident('c'), Div, Num(8), ParenRight, Semicolon ]
```

Tokens also often contain information, such as source location (file, line, column). We ignore this in this course for simplicity.

Parser: Reconstructing sentences

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

Input:

```
[ Ident('print'), ParenLeft,  
  ParenLeft, Ident('a'), Plus,  
  Num(3), ParenRight, Times,  
  Ident('b'), Plus, ParenLeft,  
  Ident('a'), Plus, Num(3),  
  ParenRight, Times,  
  Ident('c'), Div, Num(8),  
  ParenRight, Semicolon ]
```

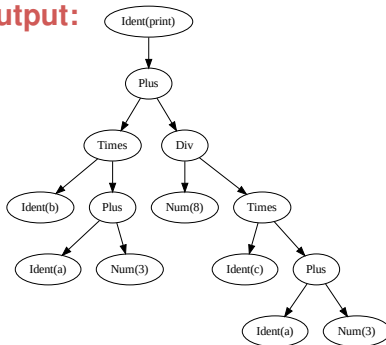
Parser: Reconstructing sentences

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

Input:

```
[ Ident('print'), ParenLeft,  
  ParenLeft, Ident('a'), Plus,  
  Num(3), ParenRight, Times,  
  Ident('b'), Plus, ParenLeft,  
  Ident('a'), Plus, Num(3),  
  ParenRight, Times,  
  Ident('c'), Div, Num(8),  
  ParenRight, Semicolon ]
```

Output:



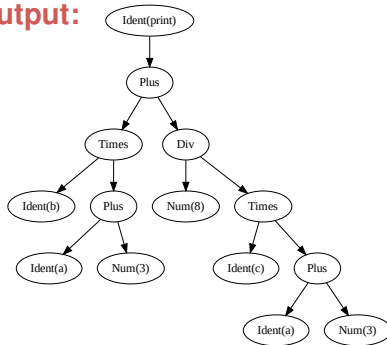
Parser: Reconstructing sentences

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

Input:

```
[ Ident('print'), ParenLeft,  
  ParenLeft, Ident('a'), Plus,  
  Num(3), ParenRight, Times,  
  Ident('b'), Plus, ParenLeft,  
  Ident('a'), Plus, Num(3),  
  ParenRight, Times,  
  Ident('c'), Div, Num(8),  
  ParenRight, Semicolon ]
```

Output:



The **AST (Abstract Syntax Tree)** is an internal representation of a program. Most compilation phases operate on ASTs rather than text.

Generating IR

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

```
MemLoad(a => %reg0)
```

```
Set(3 => %reg1)
```

```
Add(%reg0, %reg1 => %reg2)
```

Generating IR

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

```
MemLoad(a => %reg0)
```

```
Set(3 => %reg1)
```

```
Add(%reg0, %reg1 => %reg2)
```

```
MemLoad(b => %reg3)
```

```
Mul(%reg2, %reg3 => %reg4)
```

Generating IR

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

```
MemLoad(a => %reg0)
```

```
Set(3 => %reg1)
```

```
Add(%reg0, %reg1 => %reg2)
```

```
MemLoad(b => %reg3)
```

```
Mul(%reg2, %reg3 => %reg4)
```

```
MemLoad(a => %reg5)
```

```
Set(3 => %reg6)
```

```
Add(%reg5, %reg6 => %reg7)
```

Generating IR

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

```
MemLoad(a => %reg0)  
Set(3 => %reg1)  
Add(%reg0, %reg1 => %reg2)
```

```
MemLoad(b => %reg3)  
Mul(%reg2, %reg3 => %reg4)
```

```
MemLoad(a => %reg5)  
Set(3 => %reg6)  
Add(%reg5, %reg6 => %reg7)
```

```
MemLoad(c => %reg8)  
Mul(%reg7, %reg8 => %reg9)
```

Generating IR

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

```
MemLoad(a => %reg0)  
Set(3 => %reg1)  
Add(%reg0, %reg1 => %reg2)  
  
MemLoad(b => %reg3)  
Mul(%reg2, %reg3 => %reg4)  
  
MemLoad(a => %reg5)  
Set(3 => %reg6)  
Add(%reg5, %reg6 => %reg7)  
  
MemLoad(c => %reg8)  
Mul(%reg7, %reg8 => %reg9)  
  
Set(8 => %reg10)  
Div(%reg9, %reg10 => %reg11)
```

Generating IR

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

```
MemLoad(a => %reg0)  
Set(3 => %reg1)  
Add(%reg0, %reg1 => %reg2)  
  
MemLoad(b => %reg3)  
Mul(%reg2, %reg3 => %reg4)  
  
MemLoad(a => %reg5)  
Set(3 => %reg6)  
Add(%reg5, %reg6 => %reg7)  
  
MemLoad(c => %reg8)  
Mul(%reg7, %reg8 => %reg9)  
  
Set(8 => %reg10)  
Div(%reg9, %reg10 => %reg11)  
  
Add(%reg4, %reg11 => %reg12)
```

Generating IR

```
print( (a+3) *b + (a+3)*c / 0x8 );
```

```
MemLoad(a => %reg0)
Set(3 => %reg1)
Add(%reg0, %reg1 => %reg2)

MemLoad(b => %reg3)
Mul(%reg2, %reg3 => %reg4)

MemLoad(a => %reg5)
Set(3 => %reg6)
Add(%reg5, %reg6 => %reg7)

MemLoad(c => %reg8)
Mul(%reg7, %reg8 => %reg9)

Set(8 => %reg10)
Div(%reg9, %reg10 => %reg11)

Add(%reg4, %reg11 => %reg12)

Call('print', %reg12)
```

Note: IR is often represented by an AST. Here we use the textual representation for clarity.

Optimizing IR

```
MemLoad(a => %reg0)
Set(3 => %reg1)
Add(%reg0, %reg1 => %reg2)
```

```
MemLoad(b => %reg3)
Mul(%reg2, %reg3 => %reg4)
```

```
MemLoad(a => %reg5)
Set(3 => %reg6)
Add(%reg5, %reg6 => %reg7)
```

```
MemLoad(c => %reg8)
Mul(%reg7, %reg8 => %reg9)
```

```
Set(8 => %reg10)
Div(%reg9, %reg10 => %reg11)
```

```
Add(%reg4, %reg11 => %reg12)
Call('print', %reg12)
```

Optimizing IR

```
MemLoad(a => %reg0)  
Set(3 => %reg1)  
Add(%reg0, %reg1 => %reg2)
```

```
MemLoad(b => %reg3)  
Mul(%reg2, %reg3 => %reg4)
```

```
MemLoad(a => %reg5)  
Set(3 => %reg6)  
Add(%reg5, %reg6 => %reg7)
```

```
MemLoad(c => %reg8)  
Mul(%reg7, %reg8 => %reg9)
```

```
Set(8 => %reg10)  
Div(%reg9, %reg10 => %reg11)
```

```
Add(%reg4, %reg11 => %reg12)  
Call('print', %reg12)
```

Identical code blocks (after register renaming)

Optimizing IR

```
MemLoad(a => %reg0)
Set(3 => %reg1)
Add(%reg0, %reg1 => %reg2)

MemLoad(b => %reg3)
Mul(%reg2, %reg3 => %reg4)

MemLoad(c => %reg8)
Mul(%reg2, %reg8 => %reg9)

Set(8 => %reg10)
Div(%reg9, %reg10 => %reg11)

Add(%reg4, %reg11 => %reg12)
Call('print', %reg12)
```

Optimizing IR

```
MemLoad(a => %reg0)  
Set(3 => %reg1)  
Add(%reg0, %reg1 => %reg2)
```

```
MemLoad(b => %reg3)  
Mul(%reg2, %reg3 => %reg4)
```

```
MemLoad(c => %reg8)  
Mul(%reg2, %reg8 => %reg9)
```

```
Set(8 => %reg10)  
Div(%reg9, %reg10 => %reg11)
```

```
Add(%reg4, %reg11 => %reg12)  
Call('print', %reg12)
```

Division is expensive.
Division by power-of-two can be
implemented by a right-shift.

Optimizing IR

```
MemLoad(a => %reg0)  
Set(3 => %reg1)  
Add(%reg0, %reg1 => %reg2)
```

```
MemLoad(b => %reg3)  
Mul(%reg2, %reg3 => %reg4)
```

```
MemLoad(c => %reg8)  
Mul(%reg2, %reg8 => %reg9)
```

```
Set(3 => %reg10)  
ShiftRight(%reg9, %reg10 => %reg11)
```

```
Add(%reg4, %reg11 => %reg12)  
Call('print', %reg12)
```

Division is expensive.
Division by power-of-two can be
implemented by a right-shift.

Optimizing IR

```
MemLoad(a => %reg0)
Set(3 => %reg1)
Add(%reg0, %reg1 => %reg2)

MemLoad(b => %reg3)
Mul(%reg2, %reg3 => %reg4)

MemLoad(c => %reg8)
Mul(%reg2, %reg8 => %reg9)

Add(%reg4, %reg11 => %reg12)
Call('print', %reg12)
```

Backend typically includes more optimizations, e.g.:

- Using RISC instructions
 - Fuse load/store with operation
 - Use complex operations (popcnt)
 - Auto vectorization ...
- Instruction reordering

Optimizing IR

```
MemLoad(a => %reg0)
Set(3 => %reg1)
Add(%reg0, %reg1 => %reg2)

MemLoad(b => %reg3)
Mul(%reg2, %reg3 => %reg4)

MemLoad(c => %reg8)
Mul(%reg2, %reg8 => %reg9)

Add(%reg4, %reg11 => %reg12)
Call('print', %reg12)
```

Backend typically includes more optimizations, e.g.:

- Using RISC instructions
 - Fuse load/store with operation
 - Use complex operations (popcnt)
 - Auto vectorization ...
- Instruction reordering

Optimizing IR

```
MemLoad(a => %reg0)
Set(3 => %reg1)
Add(%reg0, %reg1 => %reg2)

MemLoad(b => %reg3)
Mul(%reg2, %reg3 => %reg4)

MemLoad(c => %reg8)
Mul(%reg2, %reg8 => %reg9)

Add(%reg4, %reg11 => %reg12)
Call('print', %reg12)
```

Backend typically includes more optimizations, e.g.:

- Using RISC instructions
 - Fuse load/store with operation
 - Use complex operations (popcnt)
 - Auto vectorization ...
- Instruction reordering